Binary trees follow up

Student booklet • Learn, draw, trace and practise

Name: _____ Date: _____

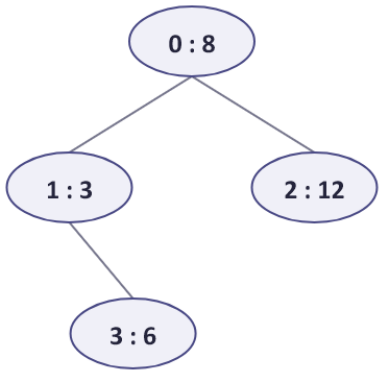
This booklet contains the same tasks as the worksheet. Use either copy. Keep indexes and data visibly different in every drawing. Attempt a task before checking the worked code with your teacher.

Learn before Task 1

A tree we can draw

Insert 8, 3, 12, 6 in that order.
Start at the root for each new value.
Smaller goes left; larger goes right.

Where does 6 go? Why?



Node labels: index : data

Task 1 • Draw a fresh sketch after each insertion

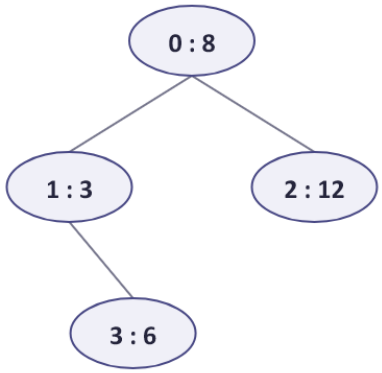
3

Slide 3 A tree we can draw

The whole subtree matters

Each node has at most two children.
A leaf has no children.
A subtree includes its root and everything below it.

The BST rule applies to the entire subtree.



Node labels: index : data

Binary trees follow up • 2D array practice

4

Slide 4 The whole subtree matters

Task 1 Draw and explain

Work on paper. An ordered binary tree is also called a binary search tree (BST). Use smaller values on the left and larger values on the right. All values here are distinct.

A. Insert 40, 20, 60, 30 in that order. Draw a new sketch after each insertion. Label each left or right link L or R.

Sketch / working space

B. On your final sketch, circle the root, underline the leaves, and loop around the whole subtree rooted at 20.

C. Explain why 30 belongs to the left subtree of 40 even though it is a right child. Does every binary tree obey the BST ordering rule?

_____

_____

_____

Learn before Task 2

A pointer stores an index

Columns: 0 = left, 1 = data, 2 = right.
-1 means there is no child.
RootPointer is the index of the root.

Index	Left	Data	Right
0	1	8	2
1	-1	3	3
2	-1	12	-1
3	-1	6	-1

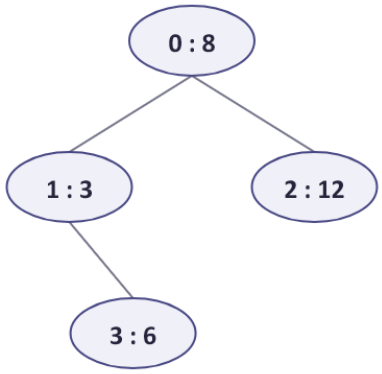
Binary trees follow up • 2D array practice 5

Slide 5 A pointer stores an index

Following a pointer in two steps

TreeArray(1, 2) = 3
Read the right pointer in row 1.

TreeArray(3, 1) = 6
Read the data in row 3.



Node labels: index : data

Task 2 • Complete the table, then reconstruct a different tree 6

Slide 6 Following a pointer in two steps

Task 2 Link the picture to storage

Keep your Task 1 tree. Allocate rows in insertion order: 40 at index 0, 20 at 1, 60 at 2 and 30 at 3. A pointer stores a ROW INDEX. -1 means no child. Columns are 0 = left, 1 = data, 2 = right.

A. Fill every blank. RootPointer = _____. Both missing child pointers must be -1.

Index	Left (0)	Data (1)	Right (2)
0		40	
1		20	
2		60	
3		30	

B. Start at row 0 and follow its left pointer, then that row's right pointer. Write the indexes visited and the data values seen.

_____

_____

C. Explain the difference between TreeArray(1, 2) and TreeArray(TreeArray(1, 2), 1). Work out both values.

_____

_____

_____

D. A different tree has RootPointer = 2 and the table below. Draw it, with each node labelled index : data. Never assume the root is row 0.

Index	Left (0)	Data (1)	Right (2)
0	-1	9	-1
1	-1	4	-1
2	1	7	0

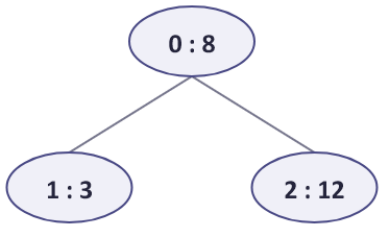
Sketch / working space

Learn before Task 3

One job for each call

InOrder(p) means:
output the subtree whose root is at index p.

Each call remembers its own p
and where to continue after its child returns.



Node labels: index : data

Slide 7 One job for each call

The empty subtree

When p = -1, return immediately.
Read no array cell. Output nothing.

A leaf still has two empty subtrees.

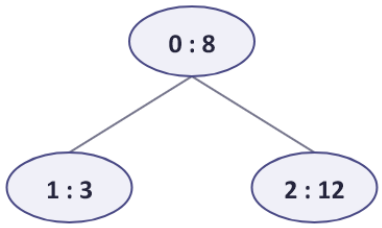
```
If p = -1 Then
    Exit Sub
End If
```


Slide 8 The empty subtree

The in-order recipe

For the subtree at p:

1. If p = -1, return.
2. Do the whole left subtree.
3. Output this node's data.
4. Do the whole right subtree.
5. Return to the caller.



Node labels: index : data

Slide 9 The in-order recipe

A leaf takes three calls

Start InOrder(1), where data = 3.
Its two child pointers are -1.

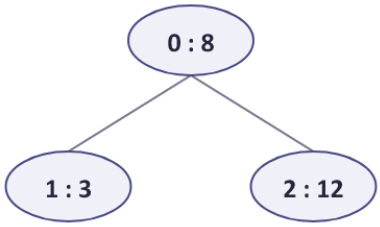
Event	Action	Output
1	Call InOrder(-1) on the left	—
2	Empty call returns to p=1	—
3	Output TreeArray(1, 1)	3
4	Call InOrder(-1) on the right	—
5	Empty call returns; leaf returns	—

Slide 10 A leaf takes three calls

The root waits

Start InOrder(0).
It calls InOrder(1).
The root call waits before its output.

After the left call finishes,
p=0 resumes and outputs 8.



Node labels: index : data

Slide 11 The root waits

A complete three-node trace

All arguments below are indexes.

Stage	What happens	Output so far
Left subtree	1 calls -1; outputs 3; calls -1; returns	3
Resume root 0	Output 8, then call InOrder(2)	3, 8
Right subtree	2 calls -1; outputs 12; calls -1; returns	3, 8, 12
Resume root 0	Return to Main	3, 8, 12

Slide 12 A complete three-node trace

Task 3 Act out calls and returns

Use the SMALL tree below. RootPointer = 1. This is a binary tree but deliberately not a BST. Follow pointers rather than sorting the values. Draw the tree before tracing.

Index	Left (0)	Data (1)	Right (2)
0	-1	15	-1
1	2	25	0
2	-1	10	-1

Sketch / working space

A. Use the numbered rules below. Each call has its own p and place to resume. On a new call, put a paper frame on top of the waiting calls. On return, remove it and resume the caller.

```
1  If p = -1, return immediately.
2  Call InOrder(left pointer of row p).
3  Output the data in row p.
4  Call InOrder(right pointer of row p).
5  Return to the caller.
```

B. Complete the trace on the next page. The first two events are done. A return from -1 produces no output. Use indexes inside InOrder(...).

Task 3 continued

Event	Active p	What happens / where next?	Output added
1	1	Call InOrder(2); p=1 waits at rule 3	—
2	2	Call InOrder(-1); p=2 waits at rule 3	—
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			

C. Just before outputting the left leaf: what is the active p, what call is waiting underneath it, and at which rule will that waiting call resume?

_____

_____

D. Write the complete output. Explain why returning from the left subtree does not end the whole traversal.

_____

_____

_____

Learn before Task 4

Your turn to complete the code

Use the recipe to fill Task 4.

Check for -1 before any array access.
Pass a child's index to each call.
Output the data between those calls.

Meaning	Column
Left child index	0
Node data	1
Right child index	2

Task 4 • Explain each gap as well as filling it 14

Slide 14 Your turn to complete the code

Task 4 Complete a recursive procedure

Return to the Task 1 tree and your Task 2A table. This version accepts -1 and checks it before reading the array. Complete the five gaps without looking at the full solution.

```
Sub InOrder(ByVal p As Integer)
    If p = (1)________ Then
        (2)________
    End If
    InOrder(TreeArray(p, (3)________))
    Console.WriteLine(TreeArray(p, (4)________))
    InOrder(TreeArray(p, (5)________))
End Sub
```

A. Explain what each of the five filled gaps does.

_____

_____

_____

_____

B. Which call starts the whole traversal? What happens if RootPointer = -1?

_____

_____

C. Predict the output for the Task 1 tree. Then draw the waiting calls when the procedure is about to output 30. Label each frame with p and its next action.

Sketch / working space

D. A student passes TreeArray(p, 1) to the recursive call. Explain the mistake and the likely effect.

_____

_____

Learn before Task 5

Two checks for broken code

Safety: does the -1 check happen first?
Order: is the output between the calls?

```
' Check the order of these actions
Check for -1
Call left subtree
Output current data
Call right subtree
```

Task 5A • Explain both bugs before repairing them 16

Slide 16 Two checks for broken code

Task 5 Repair and transfer

Use the Task 1 tree. Check the base case and the position of the output separately.

A. This procedure contains two independent mistakes. Identify each, explain its effect, and rewrite the corrected procedure below.

```
Sub Broken(ByVal p As Integer)
    Console.WriteLine(TreeArray(p, 1))  ' line A
    If p = -1 Then Exit Sub            ' line B
    Broken(TreeArray(p, 0))
    Broken(TreeArray(p, 2))
End Sub
```

Sketch / working space

B. Optional, after your teacher checks A: write PostOrder(p). It must accept a node index, safely handle -1, visit the left subtree, visit the right subtree, then output this node. Use only the 2D array.

Sketch / working space

C. Predict the post-order output for Task 1. Explain why the root appears last.

_____

_____

Learn before Task 6

Testing one idea at a time

Predict first, then run each test.
Use the supplied table; no insertion code needed.

Test	What it checks
Whole tree	Links and return order
A smaller subtree	The parameter sets the starting root
A leaf	Two empty child calls
-1	Safe empty-tree handling

Slide 17 Testing one idea at a time

Task 6 Predict then run

Open Binary_Trees_Follow_Up_Starter.vb in a VB.NET console project. Save your version as BinaryTreePractice. The table is supplied so you can focus on recursion. Do not write an insertion routine.

A. Replace the TODO in InOrder with your completed Task 4 code. Before running, fill the prediction column. Then uncomment one test call at a time, run, and record the actual output. Write “no output” where appropriate.

Test call	Prediction	Actual output
InOrder(RootPointer)		
InOrder(1)		
InOrder(2)		
InOrder(-1)		

B. Put a breakpoint on the output line. Step through the main tree. At data 30, record p and the waiting calls. Explain where execution goes after the call for node 30 finishes.

_____

_____

_____

C. Change TreeArray(3, 1) from 30 to 35. Predict the new output before running. Why do the pointers stay the same? Restore 30 afterwards.

_____

_____

D. If ready: add your PostOrder procedure from Task 5 and test the whole tree, a leaf and -1. Record predictions first.

_____

_____

Exit check: A tree has root data 8 at index 2 and only a left child, data 5 at index 0. Draw it. Give the first call, the in-order output, and the point where the root call resumes after its left call.

Sketch / working space

Optional record translation

Complete this only after Task 6. These records hold fields only. Their left and right pointers are still integer indexes. No object methods are needed.

```
Structure NodeRecord
    Public LeftPointer As Integer
    Public Data As Integer
    Public RightPointer As Integer
End Structure
Dim Nodes(3) As NodeRecord
```

A. Write a loop that sets LeftPointer and RightPointer to -1 for every row. B. Copy the Task 2A data and pointers into Nodes. C. Rewrite your working InOrder using Nodes. Predict and test the same four cases from Task 6.

Sketch / working space

Reference: Nodes(p).Data reads the data field in record p. The recursion and the -1 base case stay the same.